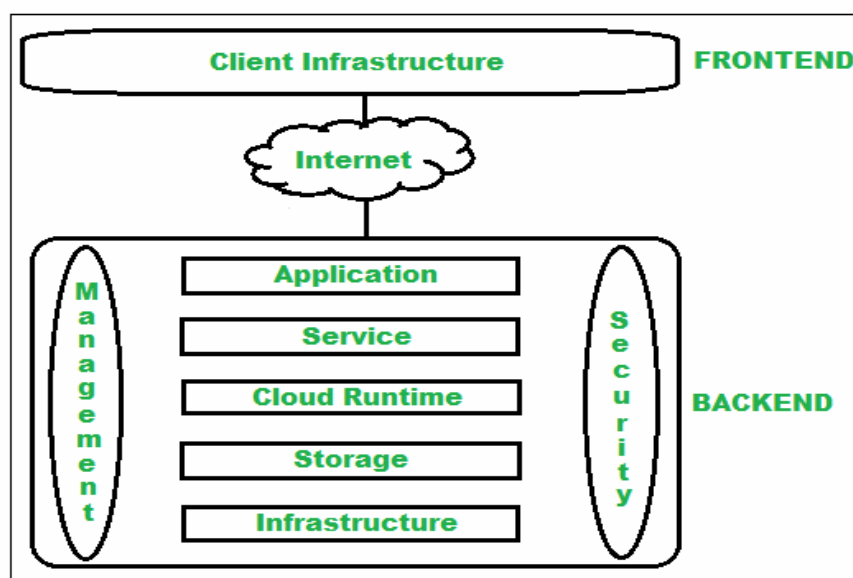
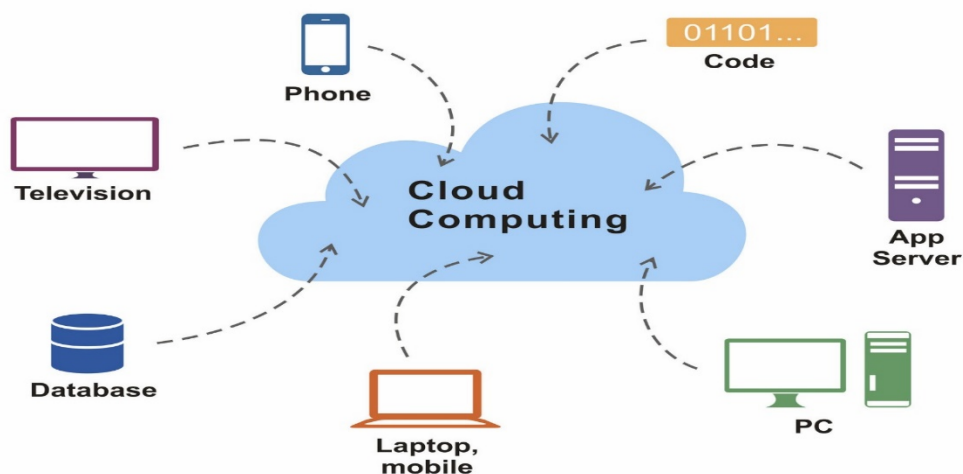


Cloud Computing : Cloud computing is the delivery of computing services—such as servers, storage, databases, networking, software, analytics, and AI—over the internet (“the cloud”) instead of using local computers or on-premises servers.

NIST (National Institute of Standards and Technology) Definition:

Cloud computing enables on-demand network access to a shared pool of configurable computing resources that can be rapidly provisioned and released with minimal management effort.



NIST defines cloud computing using 5 characteristics, 3 service models, and 4 deployment models.

NIST Cloud Computing Model Includes

1. Five Essential Characteristics:

- On-Demand Self-Service – Users can provision resources automatically.
- Broad Network Access – Services accessible via standard networks and devices.
- Resource Pooling – Provider resources serve multiple users (multi-tenancy).
- Rapid Elasticity & Scalability – Resources scale up or down quickly.
- Measured Service – Pay only for what is used (*Pay-As-You-Go Model: The pay-only-for-use model is a cloud pricing model in which users pay only for the computing resources they actually consume, such as storage, processing power, bandwidth, or software usage, for the time they are used*).

2. Three Service Models

a) **Infrastructure as a Service (IaaS)**: Provides virtualized computing resources.

Examples: AWS EC2 (Elastic Compute Cloud), Microsoft Azure VM, Google Compute Engine

User Manages: OS, applications, data

Provider Manages: Hardware, networking

b) **Platform as a Service (PaaS)**: Provides a platform for application development.

Examples: Google App Engine, Azure App Services

User Manages: Applications, data

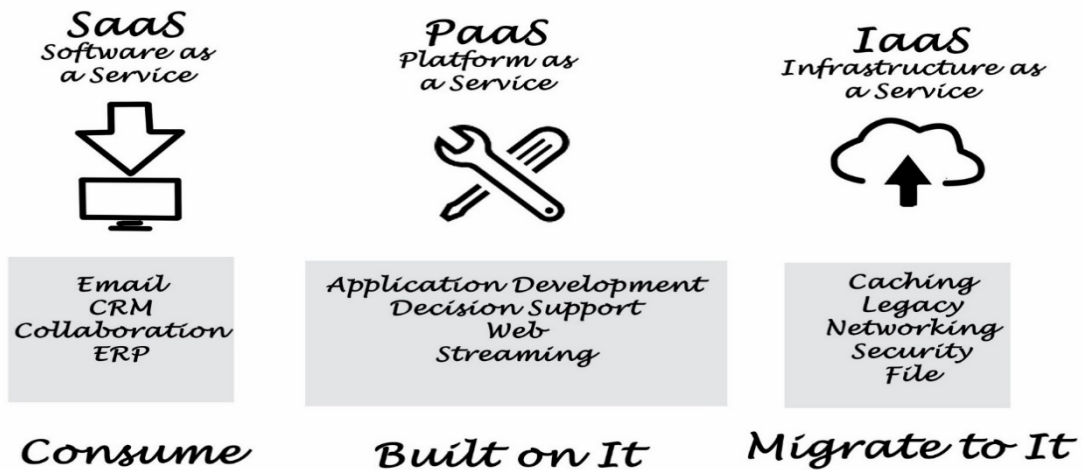
Provider Manages: OS, runtime, middleware (Bridge b/w OS and applications, infrastructure)

c) **Software as a Service (SaaS)**: Provides ready-to-use software over the internet.

Examples: Gmail, Google Docs, Microsoft 365

User Manages: Only data and usage

Provider Manages: Everything else



Other Important Cloud Services

Storage as a Service

- Cloud-based data storage
- Examples: Amazon S3, Google Cloud Storage

Database as a Service (DBaaS)

- Managed databases
- Examples: Amazon RDS, Azure SQL Database

Network as a Service (NaaS)

- Networking resources via cloud
- Examples: Virtual networks, VPN

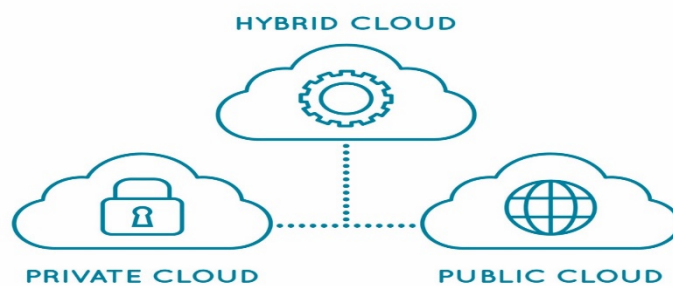
Security as a Service

- Identity, firewall, threat detection
- Examples: Cloud IAM, WAF

3. Four Deployment Models

- Public Cloud:
 - Owned by third-party providers
 - Shared infrastructure
 - Cost-effective and scalable
 - **Examples:** AWS, Azure, GCP(Google Cloud Platform)
- Private Cloud:

- Dedicated to a single organization
- Higher security and control
- Higher cost
- **Example:** On-premises cloud
- Hybrid Cloud:
 - Combination of public and private clouds
 - Sensitive data stays private, others on public cloud
 - Greater flexibility



-
- Community Cloud
 - Shared by multiple organizations
 - Supports a specific community with common concerns (security, compliance, mission)
 - May be managed internally or by a third party

Importance of NIST in Cloud Computing

- Provides standard terminology
- Ensures interoperability
- Helps in security and compliance
- Used for academic syllabus, exams, and certifications

Characteristics:

- **On-demand access** – Use resources whenever you need them
- **Scalability** – Increase or decrease resources easily
- **Pay-as-you-go** – Pay only for what you use
- **High availability** – Reliable services with backups and redundancy

- **Remote access** – Access from anywhere with the internet

Advantages of Cloud Computing

- Cost-effective (no hardware purchase)
- Scalability
- High availability
- Easy maintenance
- Global access

Disadvantages of cloud computing:

- Security risks
- Internet dependency
- Vendor lock-in
 - Use of **proprietary services and APIs**
 - Incompatible data formats
 - High data migration costs
 - Long-term contracts
 - Lack of standardization across cloud platforms
- Limited control
- Compliance and legal issues
- Possible downtime

Applications of Cloud Computing

- E-commerce websites
- Online education platforms
- Banking & finance
- Healthcare systems
- Big Data & AI
- Disaster recovery

Cloud Security

- **Confidentiality** – Data protection
- **Integrity** – Data accuracy

- **Availability** – Data accessible anytime
- **Authentication & Authorization**
- **Encryption** (Data at rest & in transit)

Evolution of Cloud Computing

1. Mainframe Computing (1950s–1960s)

- Centralized computing using **mainframe computers**
- Multiple users shared computing power via terminals
- Very expensive and limited access

Foundation of shared computing

2. Client–Server Computing (1970s–1980s)

- Processing split between **clients (PCs)** and **servers**
- Better performance and user interaction
- Required dedicated infrastructure and maintenance

Decentralization of computing

3. Internet & Web-Based Computing (1990s)

- Growth of the **Internet** enabled remote access
- Web applications replaced many desktop apps
- Introduced online services and e-commerce

Global connectivity

4. Virtualization (Early 2000s)

- Single physical server runs **multiple virtual machines**
- Improved resource utilization
- Reduced hardware costs

Key technology behind cloud computing

5. Grid & Utility Computing (Early 2000s)

- Computing resources shared across multiple systems
- **Pay-per-use** concept introduced
- Used mainly for scientific research

Precursor to cloud services (*means it is any earlier computing technology or model that contributed to the development of modern cloud computing, such as virtualization, grid computing, and utility computing*).

6. Cloud Computing Emergence (Mid-2000s)

- Amazon Web Services (AWS) launched (2006)
- On-demand access to computing resources
- Scalability and elasticity introduced

Birth of modern cloud computing

7. Service Models (2010s)

- **IaaS** – Infrastructure as a Service
- **PaaS** – Platform as a Service
- **SaaS** – Software as a Service

Standardization of cloud offerings

8. Modern Cloud & Advanced Technologies (2020s)

- **Hybrid & Multi-cloud**
- **Serverless computing**
- **AI/ML, Big Data, IoT integration**
- **Edge computing**

Highly intelligent & distributed cloud

Cloud Computing vs Traditional IT :

Feature	Cloud Computing	Traditional IT
Infrastructure Ownership	Owned & managed by cloud service providers (AWS, Azure, GCP)	Owned and managed by the organization
Cost Model	Pay-as-you-go (operational expense)	High upfront capital investment (CapEx)
Scalability	Highly scalable on demand	Limited; requires hardware purchase
Deployment Speed	Minutes to hours	Weeks to months

Maintenance	Handled by provider	Requires in-house IT team
Accessibility	Access from anywhere via internet	Mostly limited to on-premises location
Security	Shared responsibility model	Fully organization's responsibility
Disaster Recovery	Built-in, faster recovery	Expensive and complex
Performance	Depends on internet provider	& Local performance, limited reach
Upgrades	Automatic and frequent	Manual and time-consuming

Advantages

Cloud Computing

- Lower initial cost
- High availability and reliability
- Automatic updates
- Easy collaboration and remote access
- Ideal for startups & growing businesses

Traditional IT

- Full control over hardware & data
- No internet dependency
- Suitable for legacy systems
- Strong compliance control in some sectors

Use Cases

Choose Cloud Computing if:

- You need flexibility and scalability
- You want faster deployment
- You prefer lower upfront costs

- You support remote work

Choose Traditional IT if:

- You require complete control of data
- You operate in strict regulatory environments
- You run legacy or specialized hardware systems

Cloud Computing Architecture:

- **Front End:** Client devices, browsers, apps
- **Back End:** Servers, storage, databases, virtualization
- **Middleware:** Software connecting front & back end
- **Network:** Internet

Shared Responsibility Model (Cloud Computing)

Definition: The Shared Responsibility Model explains how security and operational responsibilities are divided between the cloud service provider and the cloud customer.

Both parties must work together to ensure a secure cloud environment.

Core Principle: The cloud provider secures the cloud infrastructure, while the customer is responsible for securing what they put into the cloud.

Responsibilities of Cloud Provider:

("Security of the Cloud")

- Physical data centers
- Servers and storage hardware
- Networking infrastructure
- Virtualization layer
- Physical security and availability

Responsibilities of Cloud Customer:

("Security in the Cloud")

- Data security and classification

- Identity and access management (IAM)
- Application security
- Operating system configuration (mainly in IaaS)
- Network configuration (firewalls, security groups)
- Compliance with policies

Shared Responsibility by Service Model

1. IaaS (Infrastructure as a Service)

Provider: Physical infrastructure, virtualization

Customer: OS, applications, data, access control

✧ Customer has maximum responsibility

2. PaaS (Platform as a Service)

Provider: Infrastructure, OS, runtime, middleware

Customer: Applications and data

✧ Responsibility is balanced

3. SaaS (Software as a Service)

Provider: Entire stack (app to hardware)

Customer: Data, users, access control

✧ Provider has maximum responsibility

Mode	Provider Responsibility	Customer Responsibility
IaaS	Hardware, virtualization, networking,	OS, apps, data
PaaS	Infra + OS + runtime	Apps, data
SaaS	Entire stack	Data, access

Why Shared Responsibility Model is Important

- Prevents security misunderstandings
- Clarifies accountability
- Reduces risk of data breaches
- Essential for compliance and audits

Difference between multi cloud and hybrid cloud

Introduction :

Multi-cloud and hybrid cloud are two concepts that have become increasingly popular in the world of cloud computing.

A multi-cloud strategy involves using multiple cloud computing services from different cloud providers, rather than relying on a single provider for all services. This approach can offer greater flexibility, resilience, and cost-effectiveness by leveraging the unique strengths of different cloud platforms.

On the other hand, a hybrid cloud strategy involves using a combination of both on-premises and cloud-based infrastructure to support business operations. This can allow for greater control and security over sensitive data while still taking advantage of the scalability and flexibility of cloud computing.

Both multi-cloud and hybrid cloud approaches can offer significant benefits for businesses, but also require careful planning and management to ensure optimal performance and security.

In this article, we will see how Does [Multi-Cloud](#) Differ from [Hybrid Cloud](#). Due to cloud computing technology, the IT sector is still booming. Despite the fact that the game-changing technology was released more than a decade ago, businesses are reaping its benefits of it in a variety of ways. The cloud has provided more than simply data storage and security.

Initially, the IT industry began to recognize private cloud infrastructures that could only support a company's data and workload. With the passage of time, it became clear that the cloud-based solution had matured and had been made public and controlled by third-party corporations such as Amazon Web Services (AWS), Google Cloud, and Microsoft. Hybrid and multi-cloud infrastructure is now possible in the cloud.

Multi-Cloud

The dispersion of cloud-based assets, software, and apps over many cloud environments is known as multi-cloud. With the mix-and-match technique utilized by various cloud services, the multi-cloud infrastructure is maintained specifically for a certain workload. For many businesses, the main advantage of a multi-cloud is the ability to use two or more cloud services or private clouds instead of relying on just one. Multi-cloud, on the other hand, does not allow for the linking of these numerous services.

Hybrid Cloud

The phrase "hybrid cloud" refers to a mix of private cloud on-premises and cloud services provided by third parties. In addition to traditional data centers, it is also known as a public and private cloud. It is made up of various cloud configurations in basic terms. Two cloud types could be used in the mix: two private clouds, two public clouds, or one public cloud with one private cloud.

In the business world, hybrid and multi-cloud clouds have different differences. Both names are frequently used interchangeably. Since multi-cloud computing has become the norm for many businesses, this distinction is expected to grow.

- The multi-cloud method, as is well known, makes use of a variety of cloud services that are often provided by different third-party cloud solution providers. This technique enables businesses to find a variety of cloud solutions for different departments.
- Hybrid cloud components, in contrast to multi-cloud models, usually work together. In a hybrid cloud environment, processes and data tend to mix and interconnect, as opposed to multi-cloud architectures that operate in silos.
- In practice, an application that operates on a hybrid cloud platform uses load balancing in conjunction with public cloud apps and web services. Databases and storage are housed in a private cloud structure at the same time. Resources in the cloud-based system can perform the same private and public cloud operations.
- In a multi-cloud scenario, an application could conduct all computational and networking operations on one cloud service while utilizing database services from other cloud providers. Certain apps could utilize Azure resources exclusively in multi-cloud scenarios. Other apps, on the other hand, may rely only on AWS resources. The utilization of a private and public cloud is another example. Some applications may only use resources from public clouds, while others may only use resources from private clouds.

- Aside from their differences, both cloud-based services enable organizations to provide services to clients in a more effective and productive manner.

Difference between Multi-Cloud and Hybrid Cloud

Parameters	Multi-Cloud	Hybrid Cloud
Philosophy	Multi-Cloud is a term that refers to the use of two or more public clouds, such as AWS, Azure, and Google multi-Cloud.	Hybrid clouds, such as an OpenStack private cloud and AWS, combine private and public clouds.
Multiple Public Cloud	Always required.	Based on requirements.
Concept	It is based on public clouds.	It is based on private cloud and public cloud.
Data Security	The data of users is kept safe by the public cloud provider.	The data of users is safe on the cloud, whether it is private or public.
Data Sharing	Cloud-to-cloud data transfer is possible.	Cloud data can be shared between them.
Unified Security	It is difficult to implement.	It is possible to implement.
Tool Support	Support for third-party operational tools should be prioritized.	Support for native ops tools sh

Chapter 1.2

Scalability in cloud computing allows systems to handle increased workloads by adjusting resources, categorized into vertical (scaling up) and horizontal (scaling out) methods. Vertical scaling upgrades existing hardware (e.g., adding RAM/CPU to one server), whereas horizontal scaling adds more nodes to a distributed system, offering better fault tolerance and infinite growth potential.

Vertical Scaling (Scale-Up)

- **Concept:** Increases the power of an existing machine by adding CPU, RAM, or storage.
- **Pros:** Simple to implement; no changes needed to application architecture; less networking complexity.
- **Cons:** Limited by the maximum capacity of a single machine; typically requires downtime for upgrades; creates a single point of failure.
- **Use Case:** Quick fixes for performance bottlenecks or applications that cannot be distributed.

Horizontal Scaling (Scale-Out)

- **Concept:** Adds more machines or instances (nodes) to the resource pool, distributing the load across them.
- **Pros:** High availability/fault tolerance; virtually unlimited scaling capacity; no downtime to add resources.
- **Cons:** Increased complexity in application design and data synchronization; requires load balancers.
- **Use Case:** High-traffic applications, cloud-native apps, and handling unpredictable, massive traffic.

Key Differences

Feature	Vertical Scaling (Up)	Horizontal Scaling (Out)
Approach	More Power (RAM/CPU)	More Nodes (Servers)

Availability	Lower (Single point of failure)	High (Distributed)
Downtime	Often required	No downtime
Complexity	Simple	Complex
Cost	Costly at high end	Often cheaper (commodity hardware)

Which to Choose?

- Choose **Vertical** for small-scale applications or quick, immediate fixes.
- Choose **Horizontal** for large, modern, or distributed applications requiring high uptime.

load balancing concepts, load balancing algorithms, auto-scaling principles in cloud computing

In cloud computing, load balancing and auto-scaling form a "dynamic duo" that ensures applications remain available, performant, and cost-efficient by automatically matching resources to real-time user demand

1. Load Balancing Concepts

A **load balancer** acts as a traffic director between client devices and a pool of backend servers (often called a server farm).

- **Decoupling:** Users connect to a single virtual endpoint (the load balancer), making the backend infrastructure transparent to them.
- **Health Checks:** Load balancers continuously monitor backend servers via "pings." If a server fails, traffic is automatically rerouted to healthy ones, preventing service outages.
- **Operational Layers:**
 - **Layer 4 (Transport):** Routes traffic based on IP addresses and TCP/UDP ports. It is fast and efficient for high-throughput needs.
 - **Layer 7 (Application):** Routes traffic based on content like URL paths, HTTP headers, or cookies. Ideal for microservices.

2. Load Balancing Algorithms

Algorithms determine how the load balancer selects which server should handle the next request.

- **Static Algorithms (Predefined rules):**

- **Round Robin:** Distributes requests sequentially to each server in rotation. Best for servers with identical specs.
- **Weighted Round Robin:** Assigns more traffic to powerful servers based on an assigned "weight".
- **IP Hash:** Uses a mathematical hash of the client's IP address to ensure a user consistently reaches the same server (session persistence).

- **Dynamic Algorithms (Real-time state):**

- **Least Connections:** Sends traffic to the server with the fewest active connections. Ideal for varying request lengths.
- **Least Response Time:** Prioritises the server that is currently responding the fastest.
- **Resource-Based:** Analyzes actual server metrics like current CPU or memory usage to decide where to send load.

3. Auto-Scaling Principles

Auto-scaling automatically adjusts the number of active servers in a group (Auto-Scaling Group or ASG) based on defined policies.

- **Horizontal Scaling:** The primary method in the cloud; it adds or removes entire server instances (scale out/scale in) to share the workload.
- **Scaling Strategies:**
 - **Dynamic Scaling:** Reacts to real-time metrics (e.g., "Add 2 servers if CPU > 70%"). **Target Tracking** is a common type that tries to maintain a specific metric value, like 50% CPU usage.
 - **Scheduled Scaling:** Adjusts capacity based on known time patterns, such as scaling up before a morning traffic rush.
 - **Predictive Scaling:** Uses machine learning and historical data to forecast future spikes and launch servers in advance.
- **Self-Healing:** Auto-scaling ensures high availability by automatically replacing "unhealthy" instances that fail status checks.

How They Work Together

The load balancer provides a **stable entry point**, while auto-scaling **adjusts the pool** behind it. When auto-scaling launches a new instance, it automatically registers it with the load balancer so it can immediately begin receiving traffic

What is auto scaling and load balancing in cloud computing?

Balancing the traffic load helps improve the performance of applications in a cloud environment. Auto scaling, by contrast, adjusts system capacity based on demand to maintain consistent performance and avoid overprovisioning resources (that is, using only what is needed.)

Load Balancing:

Load balancing is a technique used to distribute incoming network traffic across multiple servers or resources to ensure optimal utilization and prevent overloading of any single component. The primary goals of load balancing are to enhance application responsiveness, minimize downtime, and maximize resource utilization. Here are some key aspects of load balancing:

1. **Types of Load Balancing:****Static Load Balancing:** Assigns tasks to servers in a pre-determined manner, often based on fixed rules or algorithms.**Dynamic Load Balancing:** Adjusts the distribution of tasks in real-time based on the current state of the system, such as server load or response time.
2. **Load Balancing Algorithms:****Round Robin:** Distributes incoming requests equally among servers.**Least Connections:** Sends requests to the server with the fewest active connections.**Weighted Round Robin:** Assigns weights to servers based on their capacity, directing more traffic to higher-capacity servers.
3. **Benefits of Load Balancing:**Improved performance and response times.Enhanced fault tolerance and reliability.Efficient resource utilization.

Auto Scaling:

Auto scaling is a dynamic resource provisioning and management technique that automatically adjusts the number of resources allocated to an application based on changes in demand. The key objective is to maintain optimal performance and cost efficiency by scaling resources up during periods of increased demand and down during periods of reduced demand. Let's explore the fundamental aspects of auto scaling:

1. **Key Components of Auto Scaling:****Scaling Groups:** Logical groupings of instances that share the same configuration and scale together.**Launch Configuration:** Defines the specifications for instances, such as the AMI, instance type, and storage.**Scaling Policies:** Rules that dictate when to scale and by how much.
2. **Scaling Triggers:****Scheduled Scaling:** Pre-defined schedules for adjusting capacity based on anticipated changes in demand.**Dynamic Scaling:** Responds to changes in demand by automatically adjusting the number of instances.
3. **Benefits of Auto Scaling:****Cost Efficiency:** Ensures that resources are only provisioned when needed, minimizing unnecessary costs.**Improved Reliability:** Maintains performance

and availability by dynamically adapting to changing workloads.**Ease of Management:** Reduces the need for manual intervention in scaling processes.

Integration of Load Balancing and Auto Scaling:

The seamless integration of load balancing and auto scaling is crucial for building scalable and resilient applications. By combining these two techniques, organizations can achieve enhanced performance, reliability, and cost-effectiveness. Here's how they work together:

1. **Load Balancer and Auto Scaling Groups:**Auto scaling groups work in tandem with load balancers to automatically adjust the number of instances based on demand.Instances added or removed by auto scaling are registered or deregistered from the load balancer to maintain even distribution.
 2. **Dynamic Resource Allocation:**Auto scaling uses load metrics to trigger scaling events, ensuring that resources are scaled in or out dynamically to handle varying workloads.
 3. **High Availability and Fault Tolerance:**Load balancing distributes incoming traffic across multiple instances, and auto scaling maintains the desired number of instances, enhancing both availability and fault tolerance.
-

Let's delve into an analogy to understand auto scaling and load balancing using a real-life scenario - a popular restaurant during different times of the day.

Scenario: The Busy Restaurant

Imagine you own a bustling restaurant that experiences varying levels of customer traffic throughout the day. To efficiently manage the demand and ensure a seamless dining experience, you employ both auto scaling and load balancing mechanisms.

1. **Load Balancing - Front Desk Staff:Real-Life Analogy:** The front desk staff at the restaurant can be compared to a load balancer. Their role is to evenly distribute incoming customers among the available tables.**Explanation:** During peak hours, the load on the restaurant is high, and a lot of customers are walking in. The front desk staff ensures that customers are directed to various sections of the restaurant, avoiding overcrowding in one area. This helps in optimizing the space, ensuring that each table is utilized efficiently.
2. **Auto Scaling - Kitchen Staff:Real-Life Analogy:** The kitchen staff represents the auto scaling mechanism. They dynamically adjust the number of chefs based on the current workload.**Explanation:** During lunch or dinner rushes, the demand on the kitchen is high. Auto scaling in the kitchen involves adding more chefs to handle the increased orders. Conversely, during slower periods, some chefs may be given breaks or assigned other tasks, ensuring that the kitchen operates at an optimal capacity.

What is load balancing?

Load balancing is the method of distributing network traffic equally across a pool of resources that support an application. Modern applications must process millions of users simultaneously

and return the correct text, videos, images, and other data to each user in a fast and reliable manner. To handle such high volumes of traffic, most applications have many resource servers with duplicate data between them. A load balancer is a device that sits between the user and the server group and acts as an invisible facilitator, ensuring that all resource servers are used equally.

What are the benefits of load balancing?

Load balancing directs and controls internet traffic between the application servers and their visitors or clients. As a result, it improves an application's availability, scalability, security, and performance.

Application availability

Server failure or maintenance can increase application downtime, making your application unavailable to visitors. Load balancers increase the fault tolerance of your systems by automatically detecting server problems and redirecting client traffic to available servers. You can use load balancing to make these tasks easier:

- Run application server maintenance or upgrades without application downtime
- Provide automatic disaster recovery to backup sites
- Perform health checks and prevent issues that can cause downtime

Application scalability

You can use load balancers to direct network traffic intelligently among multiple servers. Your applications can handle thousands of client requests because load balancing does the following:

- Prevents traffic bottlenecks at any one server
- Predicts application traffic so that you can add or remove different servers, if needed
- Adds redundancy to your system so that you can scale with confidence

Application security

Load balancers come with built-in security features to add another layer of security to your internet applications. They are a useful tool to deal with distributed denial of service attacks, in which attackers flood an application server with millions of concurrent requests that cause server failure. Load balancers can also do the following:

- Monitor traffic and block malicious content
- Automatically redirect attack traffic to multiple backend servers to minimize impact
- Route traffic through a group of network firewalls for additional security

Application performance

Load balancers improve application performance by increasing response time and reducing network latency. They perform several critical tasks such as the following:

- Distribute the load evenly between servers to improve application performance
- Redirect client requests to a geographically closer server to reduce latency
- Ensure the reliability and performance of physical and virtual computing resources

What are load balancing algorithms?

A load balancing algorithm is the set of rules that a load balancer follows to determine the best server for each of the different client requests. Load balancing algorithms fall into two main categories.

Static load balancing

Static load balancing algorithms follow fixed rules and are independent of the current server state. The following are examples of static load balancing.

Round-robin method

Servers have IP addresses that tell the client where to send requests. The IP address is a long number that is difficult to remember. To make it easy, a Domain Name System maps website names to servers. When you enter aws.amazon.com into your browser, the request first goes to our name server, which returns our IP address to your browser.

In the round-robin method, an authoritative name server does the load balancing instead of specialized hardware or software. The name server returns the IP addresses of different servers in the server farm turn by turn or in a round-robin fashion.

Weighted round-robin method

In weighted round-robin load balancing, you can assign different weights to each server based on their priority or capacity. Servers with higher weights will receive more incoming application traffic from the name server.

IP hash method

In the IP hash method, the load balancer performs a mathematical computation, called hashing, on the client IP address. It converts the client IP address to a number, which is then mapped to individual servers.

Dynamic load balancing

Dynamic load balancing algorithms examine the current state of the servers before distributing traffic. The following are some examples of dynamic load balancing algorithms.

Least connection method

A connection is an open communication channel between a client and a server. When the client sends the first request to the server, they authenticate and establish an active connection between each other. In the least connection method, the load balancer checks which servers have the fewest active connections and sends traffic to those servers. This method assumes that all connections require equal processing power for all servers.

Weighted least connection method

Weighted least connection algorithms assume that some servers can handle more active connections than others. Therefore, you can assign different weights or capacities to each server, and the load balancer sends the new client requests to the server with the least connections by capacity.

Least response time method

The response time is the total time that the server takes to process the incoming requests and send a response. The least response time method combines the server response time and the active connections to determine the best server. Load balancers use this algorithm to ensure faster service for all users.

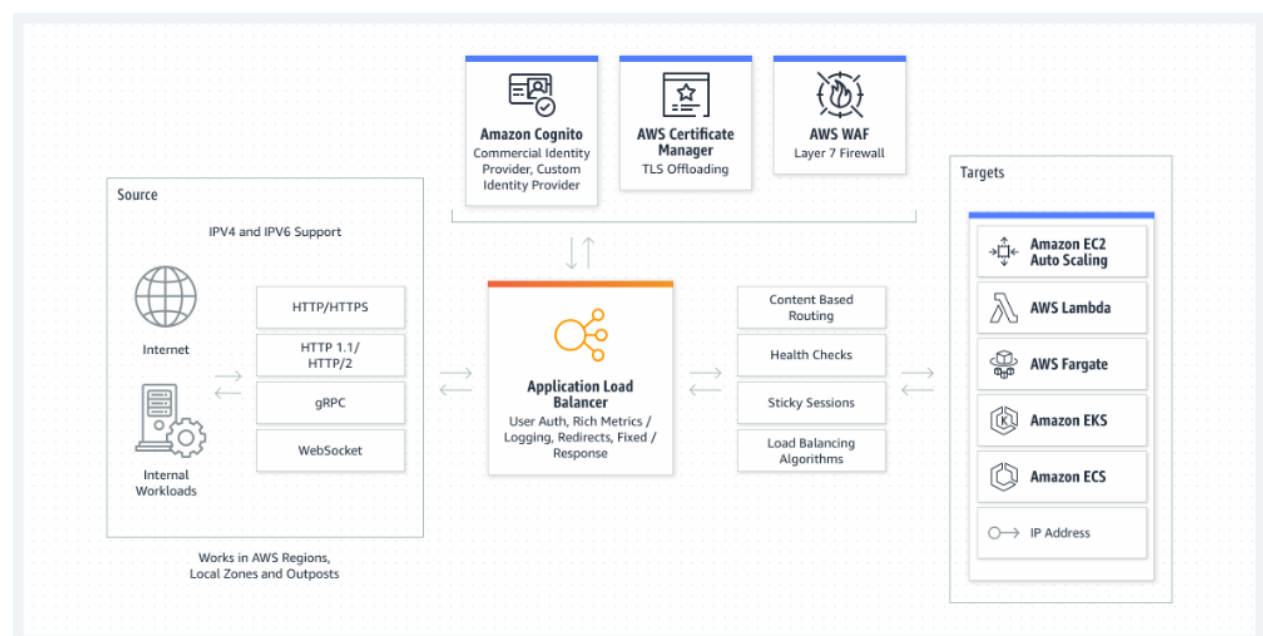
Resource-based method

In the resource-based method, load balancers distribute traffic by analyzing the current server load. Specialized software called an agent runs on each server and calculates usage of server resources, such as its computing capacity and memory. Then, the load balancer checks the agent for sufficient free resources before distributing traffic to that server.

How does load balancing work?

Companies usually have their application running on multiple servers. Such a server arrangement is called a server farm. User requests to the application first go to the load balancer. The load balancer then routes each request to a single server in the server farm best suited to handle the request.

Load balancing is like the work done by a manager in a restaurant. Consider a restaurant with five waiters. If customers were allowed to choose their waiters, one or two waiters could be overloaded with work while the others are idle. To avoid this scenario, the restaurant manager assigns customers to the specific waiters who are best suited to serve them.



What are the types of load balancing?

We can classify load balancing into three main categories depending on what the load balancer checks in the client request to redirect the traffic.

Application load balancing

Complex modern applications have several server farms with multiple servers dedicated to a single application function. Application load balancers look at the request content, such as HTTP headers or SSL session IDs, to redirect traffic.

For example, an ecommerce application has a product directory, shopping cart, and checkout functions. The application load balancer sends requests for browsing products to servers that contain images and videos but do not need to maintain open connections. By comparison, it sends shopping cart requests to servers that can maintain many client connections and save cart data for a long time.

Network load balancing

Network load balancers examine IP addresses and other network information to redirect traffic optimally. They track the source of the application traffic and can assign a static IP address to several servers. Network load balancers use the static and dynamic load balancing algorithms described earlier to balance server load.

Global server load balancing

Global server load balancing occurs across several geographically distributed servers. For example, companies can have servers in multiple data centers, in different countries, and in third-party cloud providers around the globe. In this case, local load balancers manage the application load within a region or zone. They attempt to redirect traffic to a server destination that is geographically closer to the client. They might redirect traffic to servers outside the client's geographic zone only in case of server failure.

DNS load balancing

In DNS load balancing, you configure your domain to route network requests across a pool of resources on your domain. A domain can correspond to a website, a mail system, a print server, or another service that is made accessible through the internet. DNS load balancing is helpful for maintaining application availability and balancing network traffic across a globally distributed pool of resources.

What are the types of load balancing technology?

Load balancers are one of two types: hardware load balancer and software load balancer.

Hardware load balancers

A hardware-based load balancer is a hardware appliance that can securely process and redirect gigabytes of traffic to hundreds of different servers. You can store it in your data centers and use virtualization to create multiple digital or virtual load balancers that you can centrally manage.

Software load balancers

Software-based load balancers are applications that perform all load balancing functions. You can install them on any server or access them as a fully managed third-party service.

Comparison of hardware balancers to software load balancers

Hardware load balancers require an initial investment, configuration, and ongoing maintenance. You might also not use them to full capacity, especially if you purchase one only to handle peak-time traffic spikes. If traffic volume increases suddenly beyond its current capacity, this will affect users until you can purchase and set up another load balancer.

In contrast, software-based load balancers are much more flexible. They can scale up or down easily and are more compatible with modern cloud computing environments. They also cost less to set up, manage, and use over time.

How can AWS help with load balancing?

[Elastic Load Balancing \(ELB\)](#) is a fully managed load balancing service that automatically distributes incoming application traffic to multiple targets and virtual appliances across AWS and on-premises resources. You can use it to scale modern applications without complex configurations or API gateways. You can use ELB to set up four different types of software load balancers.

- An Application Load Balancer routes traffic for HTTP-based requests.
- A Network Load Balancer routes traffic based on IP addresses. It is ideal for balancing TCP and User Datagram Protocol (UDP)-based requests.
- A Gateway Load Balancer routes traffic to third-party virtual appliances. It is ideal for incorporating a third-party appliance, such as a network firewall, into your network traffic in a scalable and easy-to-manage way.
- A Classic Load Balancer routes traffic to applications in the [Amazon EC2](#)-Classic network—a single, flat network that you share with other customers.

You can select the load balancer based on your requirements. For example, [Terminix](#), a global pest control brand, uses Gateway Load Balancer to handle 300% more throughput. [Second Spectrum](#), a company that provides artificial intelligence-driven tracking technology for sports broadcasts, uses AWS Load Balancer Controller to reduce hosting costs by 90%. [Code.org](#), a nonprofit dedicated to expanding access to computer science in schools, uses Application Load Balancer to handle a 400% spike in traffic efficiently during online coding events.

Get started with load balancing by [creating an AWS account](#) today!

Virtualization & Cloud Migration

(Exam-ready notes with clear concepts, diagrams-in-words, and key points)

1. Virtualization

Definition:

Virtualization is the technology that allows multiple virtual machines (VMs) with their own OS and applications to run on a single physical server using a **hypervisor**.

Why it's used

- Better hardware utilization
- Reduced cost (fewer physical servers)
- Faster provisioning and scalability
- Isolation and security between workloads

Key Components

- **Host Machine:** Physical server
- **Hypervisor:** Software that creates/manages VMs
- **Guest OS:** Operating system inside a VM

Types of Hypervisors

- **Type 1 (Bare Metal):** Runs directly on hardware
 - Examples: VMware ESXi, Microsoft Hyper-V, Xen
- **Type 2 (Hosted):** Runs on top of an OS
 - Examples: VirtualBox, VMware Workstation

Types of Virtualization

- **Server Virtualization** – Multiple servers on one machine
- **Storage Virtualization** – Pooling storage resources
- **Network Virtualization** – Virtual networks (VLAN, SDN)
- **Desktop Virtualization** – Virtual desktops (VDI)

2. Cloud Migration

Definition:

Cloud migration is the process of moving applications, data, and

workloads from on-premises infrastructure to the cloud (AWS, Azure, GCP) or between clouds.

Why organizations migrate

- Cost optimization (pay-as-you-go)
- High availability & disaster recovery
- Elastic scalability
- Reduced infrastructure management

3. Relationship Between Virtualization & Cloud

- **Virtualization is the foundation of cloud computing**
- Cloud providers use virtualization to:
 - Create VMs
 - Isolate customers
 - Scale resources dynamically

4. Cloud Migration Strategies (6 R's) (*Very important for exams*)

1. Rehost (Lift & Shift)

- Move apps as-is to cloud VMs
- Fast, minimal changes

2. Refactor / Re-architect

- Redesign app using cloud-native services
- High cost, high benefit

3. Replatform

- Minor optimizations without major redesign

4. Repurchase

- Switch to SaaS (e.g., CRM → Salesforce)

5. Retire

- Remove unused applications

6. Retain

- Keep some apps on-prem (compliance/latency)

5. Cloud Migration Process

1. Assessment & Planning

- Application dependency analysis
- Cost estimation

2. Choose Migration Strategy

3. Data & Application Migration

4. Testing & Validation

5. Optimization & Monitoring

6. Benefits of Cloud Migration

- Reduced IT operational costs
- Automatic updates & maintenance
- Improved performance & reliability
- Business agility and faster innovation

7. Challenges in Cloud Migration

- Data security & privacy concerns
- Downtime during migration
- Application compatibility issues
- Vendor lock-in

8. Virtualization vs Cloud (Quick Comparison)

Feature	Virtualization	Cloud Computing
----------------	-----------------------	------------------------

Scope	Technology	Service model
Ownership	On-prem	Third-party provider
Scalability	Limited	Highly elastic
Cost Model	CapEx	OpEx

Chapter 1.3

- [Paravirtualization](#) is virtualization in which the guest operating system (the one being virtualized) is aware that it is a guest and accordingly has drivers that, instead of issuing hardware commands, simply issue commands directly to the host operating system. This also includes memory and thread management as well, which usually require unavailable privileged instructions in the processor.
- [Full Virtualization](#) is virtualization in which the guest operating system is unaware that it is in a virtualized environment, and therefore hardware is virtualized by the host operating system so that the guest can issue commands to what it thinks is actual hardware, but really are just simulated hardware devices created by the host.
- [Hardware Assisted Virtualization](#) is a type of Full Virtualization where the microprocessor architecture has special instructions to aid the virtualization of hardware. These instructions might allow a virtual context to be setup so that the guest can execute privileged instructions directly on the processor without affecting the host. Such a feature set is often called a **Hypervisor**. If said instructions do not exist, Full Virtualization is still possible, however it must be done via software techniques such as **Dynamic Recompilation** where the host recompiles on the fly privileged instructions in the guest to be able to run in a non-privileged way on the host.
- There is also a combination of Para Virtualization and Full Virtualization called **Hybrid Virtualization** where parts of the guest operating system use

paravirtualization for certain hardware drivers, and the host uses full virtualization for other features. This often produces superior performance on the guest without the need for the guest to be completely paravirtualized. An example of this: The guest uses full virtualization for privileged instructions in the kernel but paravirtualization for IO requests using a special driver in the guest. This way the guest operating system does not need to be fully paravirtualized, since this is sometimes not available, but can still enjoy some paravirtualized features by implementing special drivers for the guest.

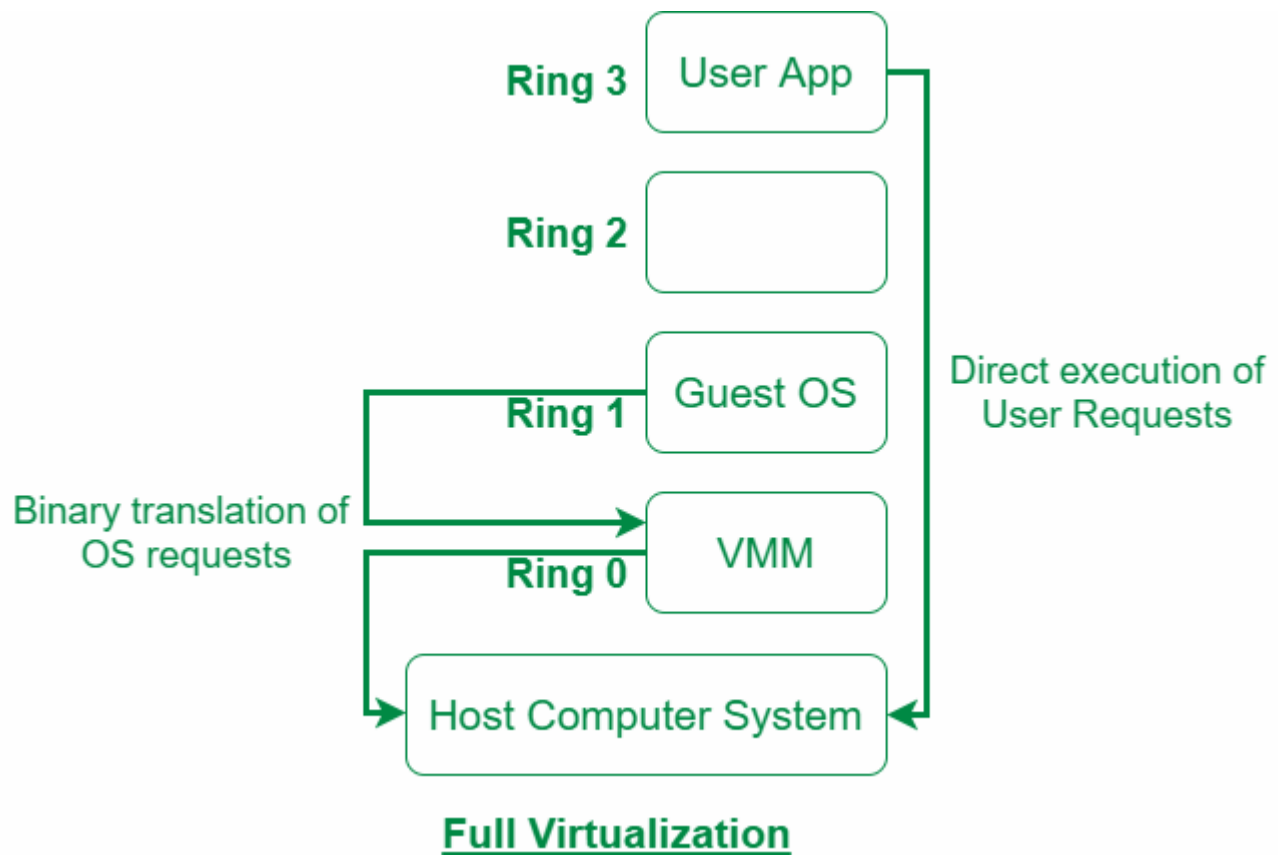
Virtualization allows one computer to function as multiple computers by sharing its resources across different environments. CPU virtualization includes full virtualization and paravirtualization. In full virtualization, the original operating system runs without knowing it's virtualized, using translation to handle system calls. Paravirtualization modifies the OS to use hypercalls instead of certain instructions, making the process more efficient but requiring changes before compiling.

In this article, we'll explore the differences between Full Virtualization and Paravirtualization in operating systems. But first, let's understand what each of these terms means.

What is Full Virtualization?

Full Virtualization was introduced by IBM in 1966. It is the first software solution for server virtualization and uses binary translation and direct approach techniques. In full virtualization, the virtual machine completely isolates the guest OS from the virtualization layer and hardware. Microsoft

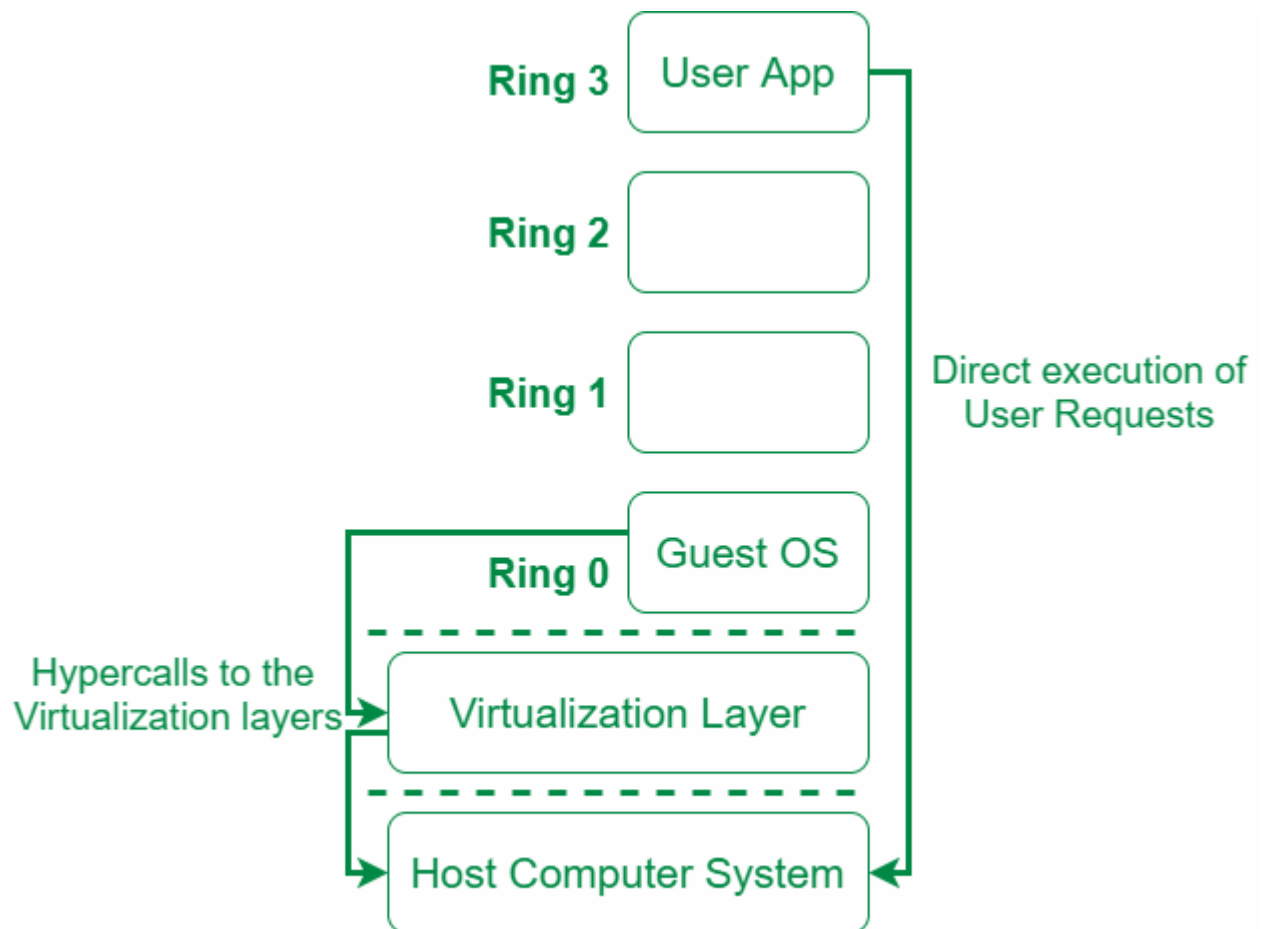
and Parallels systems are examples of full virtualization.



What is Paravirtualization?

Paravirtualization is the category of CPU virtualization which uses hypercalls for operations to handle instructions at compile time. In paravirtualization, guest OS is not completely isolated but it is partially isolated by the virtual machine from the virtualization layer and

hardware. [VMware](#) and Xen are some examples of paravirtualization.



Paravirtualization

Difference Between Full Virtualization and Paravirtualization

The difference between Full Virtualization and Paravirtualization are as follows:

S.No.	Full Virtualization	Paravirtualization
1.	In Full virtualization, virtual machines permit the execution of the instructions with the running of unmodified OS in an entirely isolated way.	In paravirtualization, a virtual machine does not implement full isolation of OS but rather provides a different API which is utilized when OS is subjected to alteration.
2.	Full Virtualization is less secure.	While the Paravirtualization is more secure than the Full Virtualization.
3.	Full Virtualization uses binary translation and a direct	While Paravirtualization uses hypercalls at compile time for

	approach as a technique for operations.	operations.
4.	Full Virtualization is slow than paravirtualization in operation.	Paravirtualization is faster in operation as compared to full virtualization.
5.	Full Virtualization is more portable and compatible.	Paravirtualization is less portable and compatible.
6.	Examples of full virtualization are Microsoft and Parallels systems.	Examples of paravirtualization are Microsoft Hyper-V, Citrix Xen, etc.
7.	It supports all guest operating systems without modification.	The guest operating system has to be modified and only a few operating systems support it.
8.	The guest operating system will issue hardware calls.	Using the drivers, the guest operating system will directly communicate with the hypervisor.
9.	It is less streamlined compared to paravirtualization.	It is more streamlined.
10.	It provides the best isolation.	It provides less isolation compared to full virtualization.

Conclusion

There are two types of virtualization methods: full and paravirtualization. Full virtualization isolates the unmodified operating system completely. In contrast, paravirtualization modifies the Operating System to work with specific APIs without complete isolation.

Brief summery of above topic-

Virtualization

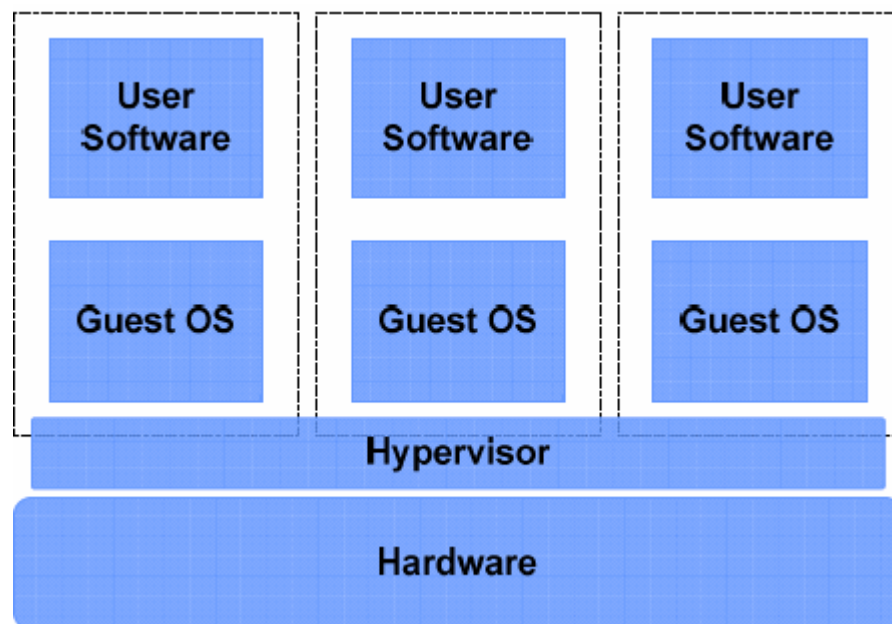
Definition

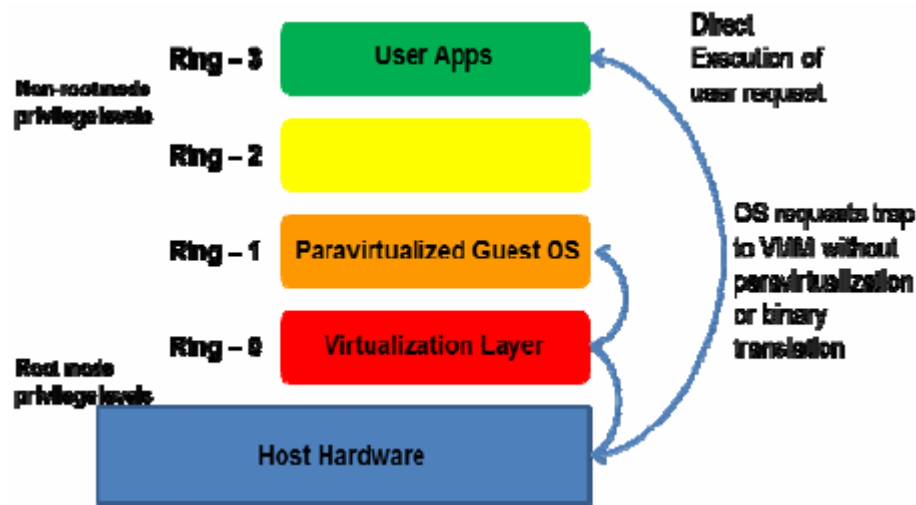
Virtualization is a technology that allows multiple operating systems to run simultaneously on a single physical machine by sharing hardware resources through a software layer called a **hypervisor (VMM – Virtual Machine Monitor)**.

Need for Virtualization

- Efficient utilization of hardware
- Reduced cost of infrastructure
- Isolation between applications
- Easy backup and recovery
- Scalability and flexibility

Types of Virtualization (Based on Technique)





1. Full Virtualization

Definition:

In full virtualization, the **hypervisor completely emulates the hardware**, and the **guest operating system runs without any modification**.

Features:

- No modification of guest OS
- Complete isolation between virtual machines
- Hardware is fully virtualized

Advantages:

- Supports unmodified OS
- High compatibility

Disadvantages:

- Performance overhead due to hardware emulation

Examples:

- VMware ESXi
- VirtualBox
- Microsoft Hyper-V

2. Para-Virtualization

Definition:

In para-virtualization, the **guest OS is modified** to communicate directly with the hypervisor using **hypercalls**.

Features:

- Guest OS is aware of virtualization
- Reduced overhead compared to full virtualization

Advantages:

- Better performance
- Efficient resource utilization

Disadvantages:

- Requires OS modification
- Limited OS support

Examples:

- Xen (Para-virtualized mode)

3. Hardware-Assisted Virtualization

Definition:

Hardware-assisted virtualization uses **CPU-provided virtualization extensions** to support virtual machines efficiently without modifying the guest OS.

Features:

- Uses processor support
- Near-native performance
- No guest OS modification

CPU Technologies:

- Intel VT-x
- AMD-V

Advantages:

- High performance
- Better security and isolation

Examples:

- KVM
- VMware
- Hyper-V

Hypervisor

Definition:

A **hypervisor** is a software/firmware layer that creates and manages virtual machines.

Types:

1. **Type-1 (Bare Metal):** Runs directly on hardware (VMware ESXi, Hyper-V)
2. **Type-2 (Hosted):** Runs on host OS (VirtualBox, VMware Workstation)

Comparison Table

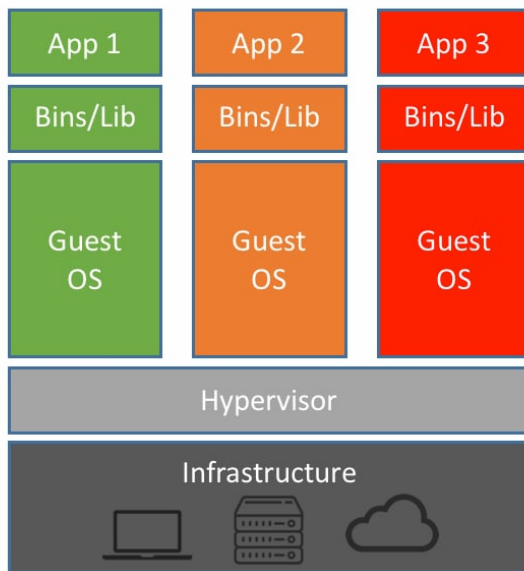
Feature	Full Virtualization	Para-Virtualization	Hardware-Assisted
Guest OS Modification	No	Yes	No
Performance	Moderate	High	Very High
Hardware Support	Optional	Not required	Required
OS Compatibility	High	Limited	High

Short Exam Points (2–3 Marks)

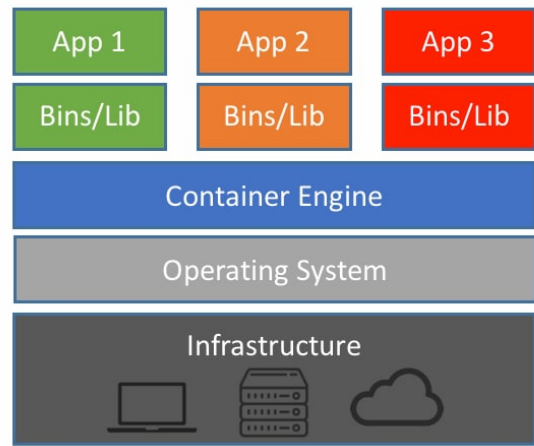
- Virtualization increases hardware utilization.
- Para-virtualization uses hypercalls.
- Hardware-assisted virtualization relies on Intel VT-x and AMD-V.
- Hypervisor controls virtual machines.

containers vs. virtual machines.

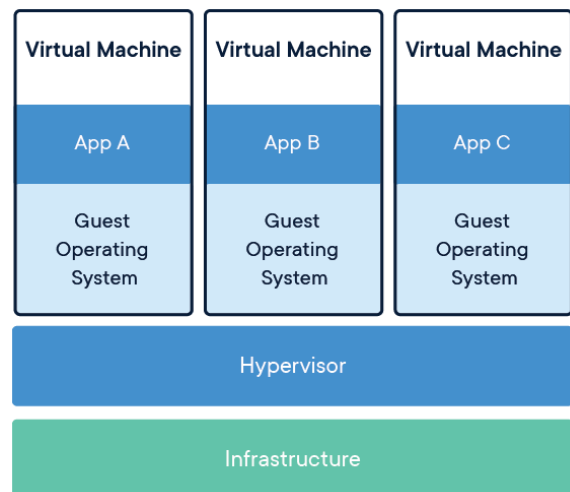
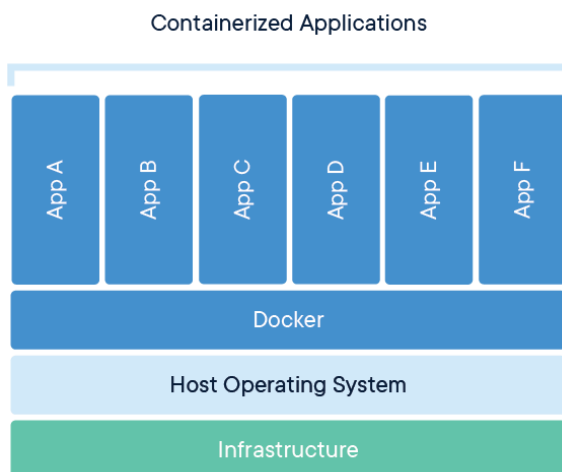
Containers vs. Virtual Machines — what's the real difference?



Machine Virtualization



Containers



Think of **VMs** as fully furnished apartments and **containers** as furnished rooms in a shared house. Both isolate apps—but they do it very differently.

Virtual Machines (VMs)

- **What they include:** App + libraries + *entire guest OS*
- **How they run:** On a **hypervisor** (VMware, Hyper-V, KVM)
- **Isolation:** Strong (hardware-level)
- **Startup time:** Minutes
- **Resource use:** Heavier (duplicate OS per VM)
- **Best for:** Running different OSes, strict security isolation, legacy apps

Example: One VM with Windows Server, another with Ubuntu—same host.

Containers

- **What they include:** App + libraries (share the **host OS kernel**)
- **How they run:** Via a container runtime (Docker, containerd) and often orchestrated by **Kubernetes**
- **Isolation:** Process-level (lighter than VMs)
- **Startup time:** Seconds
- **Resource use:** Lightweight and efficient
- **Best for:** Microservices, CI/CD, cloud-native apps, rapid scaling

Example: Many containerized apps all sharing the Linux kernel.

Side-by-side comparison

Aspect	Virtual Machines	Containers
OS	Full guest OS per VM	Share host OS kernel
Size	Large (GBs)	Small (MBs)
Speed	Slower to boot	Very fast
Isolation	Very strong	Moderate
Density	Fewer per host	Many per host
Portability	Moderate	Excellent
Common tools	VMware, Hyper-V	Docker, Kubernetes

When to choose what?

- **Choose VMs if** you need different operating systems, hard isolation, or compliance-heavy workloads.
- **Choose containers if** you want speed, scalability, and efficient resource usage for modern app stacks.
- **Hybrid is common:** Run **containers inside VMs** for the best of both worlds.